```
import pandas as pd
import numpy as np
```

```
df=pd.read_csv('/content/boston.csv')
```

```
df
```

| | CRIM | ZN | INDUS | CHAS | NOX | RM | AGE | DIS | RAD | TAX | PTRATIO | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **0** | 0.00632 | 18.0 | 2.31 | 0 | 0.538 | 6.575 | 65.2 | 4.0900 | 1 | 296.0 | 15.3 | 39 |
| **1** | 0.02731 | 0.0 | 7.07 | 0 | 0.469 | 6.421 | 78.9 | 4.9671 | 2 | 242.0 | 17.8 | 39 |
| **2** | 0.02729 | 0.0 | 7.07 | 0 | 0.469 | 7.185 | 61.1 | 4.9671 | 2 | 242.0 | 17.8 | 39 |
| **3** | 0.03237 | 0.0 | 2.18 | 0 | 0.458 | 6.998 | 45.8 | 6.0622 | 3 | 222.0 | 18.7 | 39 |
| **4** | 0.06905 | 0.0 | 2.18 | 0 | 0.458 | 7.147 | 54.2 | 6.0622 | 3 | 222.0 | 18.7 | 39 |
| **...** | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | |
| **501** | 0.06263 | 0.0 | 11.93 | 0 | 0.573 | 6.593 | 69.1 | 2.4786 | 1 | 273.0 | 21.0 | 39 |
| **502** | 0.04527 | 0.0 | 11.93 | 0 | 0.573 | 6.120 | 76.7 | 2.2875 | 1 | 273.0 | 21.0 | 39 |
| **503** | 0.06076 | 0.0 | 11.93 | 0 | 0.573 | 6.976 | 91.0 | 2.1675 | 1 | 273.0 | 21.0 | 39 |
| **504** | 0.10959 | 0.0 | 11.93 | 0 | 0.573 | 6.794 | 89.3 | 2.3889 | 1 | 273.0 | 21.0 | 39 |
| **505** | 0.04741 | 0.0 | 11.93 | 0 | 0.573 | 6.030 | 80.8 | 2.5050 | 1 | 273.0 | 21.0 | 39 |

506 rows × 14 columns

```
df.head()
```

| | CRIM | ZN | INDUS | CHAS | NOX | RM | AGE | DIS | RAD | TAX | PTRATIO | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **0** | 0.00632 | 18.0 | 2.31 | 0 | 0.538 | 6.575 | 65.2 | 4.0900 | 1 | 296.0 | 15.3 | 396.9 |
| **1** | 0.02731 | 0.0 | 7.07 | 0 | 0.469 | 6.421 | 78.9 | 4.9671 | 2 | 242.0 | 17.8 | 396.9 |
| **2** | 0.02729 | 0.0 | 7.07 | 0 | 0.469 | 7.185 | 61.1 | 4.9671 | 2 | 242.0 | 17.8 | 392.8 |
| **3** | 0.03237 | 0.0 | 2.18 | 0 | 0.458 | 6.998 | 45.8 | 6.0622 | 3 | 222.0 | 18.7 | 394.6 |
| **4** | 0.06905 | 0.0 | 2.18 | 0 | 0.458 | 7.147 | 54.2 | 6.0622 | 3 | 222.0 | 18.7 | 396.9 |

```
df.columns
```

```
Index(['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE', 'DIS', 'RAD', 'TAX',
       'PTRATIO', 'B', 'LSTAT', 'MEDV'],
```

```
        dtype='object')
```

## Defining x and y

```
x=df.drop('MEDV',axis=1)
y=df['MEDV']
```

```
x=np.array(x)
y=np.array(y).reshape(-1,1)
```

```
x
```

```
array([[6.3200e-03, 1.8000e+01, 2.3100e+00, ..., 1.5300e+01, 3.9690e+02,
        4.9800e+00],
       [2.7310e-02, 0.0000e+00, 7.0700e+00, ..., 1.7800e+01, 3.9690e+02,
        9.1400e+00],
       [2.7290e-02, 0.0000e+00, 7.0700e+00, ..., 1.7800e+01, 3.9283e+02,
        4.0300e+00],
       ...,
       [6.0760e-02, 0.0000e+00, 1.1930e+01, ..., 2.1000e+01, 3.9690e+02,
        5.6400e+00],
       [1.0959e-01, 0.0000e+00, 1.1930e+01, ..., 2.1000e+01, 3.9345e+02,
        6.4800e+00],
       [4.7410e-02, 0.0000e+00, 1.1930e+01, ..., 2.1000e+01, 3.9690e+02,
        7.8800e+00]])
```

```
y
```

```
[19.6],
[23.2],
[29.8],
[13.8],
[13.3],
[16.7],
[12. ],
[14.6],
[21.4],
[23. ],
[23.7],
[25. ],
[21.8],
[20.6],
[21.2],
[19.1],
[20.6],
[15.2],
[ 7. ],
[ 8.1],
[13.6],
[20.1],
[21.8],
[24.5],
[23.1],
[19.7],
[18.3],
[21.2],
[17.5],
[16.8],
[22.4],
[20.6],
[23.9],
[22. ],
[11.9]])
```

Spliting the data into training and testing

```
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(
    x,y,test_size=0.3,random_state=42
)
```

```
df.isnull().sum()
```

| | 0 |
|---|---|
| **CRIM** | 0 |
| **ZN** | 0 |
| **INDUS** | 0 |
| **CHAS** | 0 |
| **NOX** | 0 |
| **RM** | 0 |
| **AGE** | 0 |
| **DIS** | 0 |
| **RAD** | 0 |
| **TAX** | 0 |
| **PTRATIO** | 0 |
| **B** | 0 |
| **LSTAT** | 0 |
| **MEDV** | 0 |

**dtype:** int64

```
from sklearn.preprocessing import StandardScaler
scaler=StandardScaler()
#Fit only on training data
x_train=scaler.fit_transform(x_train)
#transform test data using same parameters
x_test=scaler.transform(x_test)
```

```
from sklearn.linear_model import Ridge
```

```
#Greek Search
from sklearn.linear_model import RidgeCV

#Try multiple alpha values
alphas=np.logspace(-3,3,7)

ridge_cv=RidgeCV(alphas=alphas,cv=10)
ridge_cv.fit(x_train,y_train)
print("Best alpha:",ridge_cv.alpha_)
```

```
Best alpha: 10.0
```

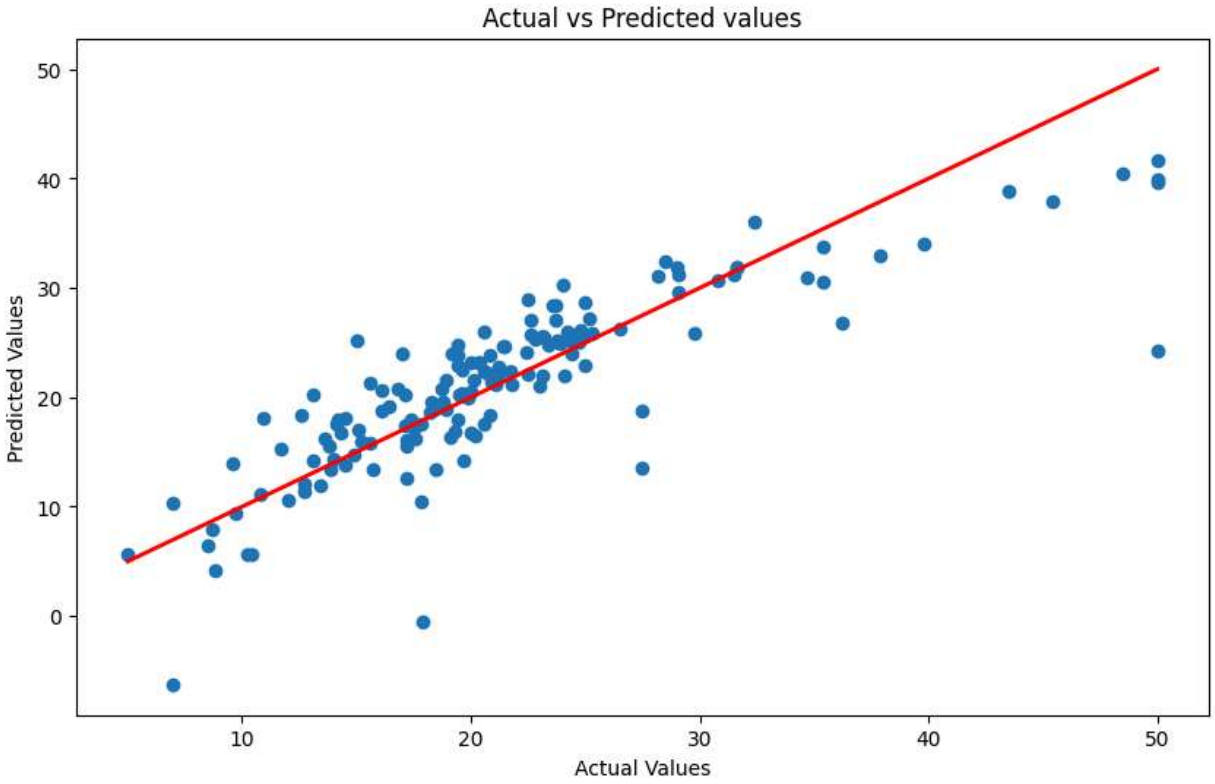
```
best_ridge=Ridge(alpha=ridge_cv.alpha_)
best_ridge.fit(x_train,y_train)
y_pred=best_ridge.predict(x_test)
```

```
from sklearn.metrics import mean_squared_error,r2_score
#MSE
mse=mean_squared_error(y_test,y_pred)
print("MSE:",mse)
#RMSE
rmse=np.sqrt(mse)
print("RMSE:",rmse)
#R2 Score
r2=r2_score(y_test,y_pred)
print("R^2 score:",r2)
```

```
MSE: 21.81124307149158
RMSE: 4.670250857447765
R^2 score: 0.7072830902371281
```

```
import matplotlib.pyplot as plt
plt.figure(figsize=(10, 6))
plt.scatter(y_test,y_pred)
plt.plot([y_test.min(),y_test.max()],[y_test.min(),y_test.max()],'r-',lw=2)
plt.title("Actual vs Predicted values")
plt.xlabel('Actual Values')
plt.ylabel('Predicted Values')
```

```
Text(0, 0.5, 'Predicted Values')
```




```
from sklearn.linear_model import Lasso
```

```
from sklearn.linear_model import LassoCV
#Try multiple alpha values
alphas=np.logspace(-3,3,7)

lasso_cv=LassoCV(alphas=alphas,cv=10)
lasso_cv.fit(x_train,y_train)
print("Best alpha:",lasso_cv.alpha_)
```

```
Best alpha: 0.001
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_coordinate_descent
  y = column_or_1d(y, warn=True)
```

```
best_ridge=Lasso(alpha=lasso_cv.alpha_)
```

```
from sklearn.metrics import mean_squared_error,r2_score
#MSE
mse=mean_squared_error(y_test,y_pred)
print("MSE:",mse)
#RMSE
rmse=np.sqrt(mse)
print("RMSE:",rmse)
#R2 Score
r2=r2_score(y_test,y_pred)
print("R^2 score:",r2)
```

```
MSE: 21.521510718492532
RMSE: 4.63912822828735
R^2 score: 0.7111714316191495
```

```
import matplotlib.pyplot as plt
plt.figure(figsize=(10,6))
plt.scatter(y_test,y_pred)
plt.plot([min(y_test),max(y_test)],[min(y_test),max(y_test)])
max_val = max(np.max(y_test), np.max(y_pred))
plt.xlim(0, max_val)
plt.ylim(0, max_val)
plt.plot([0, max_val], [0, max_val], 'b-', label='Perfect Prediction')
plt.legend()
plt.xlabel("Actual Values")
plt.ylabel("Predicted Values")
plt.title("Actual vs Predicted Values")
plt.show()
```